ChatStack

Multi-tenant RAG-as-a-Service - Project plan, starter-template inventory, and local setup

Purpose

This guide converts the supplied product specification into a practical Phase 1 implementation plan. The repository now includes a Gemini-backed API-first RAG slice: database-persisted tenants and hashed keys, document ingestion, pgvector retrieval, and chat. Authentication, rate limits, streaming, and production storage remain staged work.

Architecture at a glance

Layer	Selected technology	Role
Dashboard	React + TypeScript + Vite + React Query	Tenant dashboard, one-time API key UX, document/chat/widget screens in later slices.
API	Node.js + Express + TypeScript	Session routes, tenant control plane, tenant-scoped document/search/chat APIs.
Data	PostgreSQL + pgvector	Users, tenants, documents, chunks and 1536-dimension embeddings.
Async	Redis + BullMQ	Ingestion queue, retry/dead-letter policy, rate-limit counters.
AI	Google Gemini API	Gemini embeddings and generated RAG answers after credentials are configured.
Deployment	AWS ECS, RDS, ElastiCache, S3, CloudFront, Secrets Manager	Phase 2 production topology.

Security boundary

A dashboard user owns tenants. Each tenant receives a private server API key, stored only as a hash, and a distinct public widget key. Tenant identity must always come from authentication, not a request body or URL parameter. The public widget has an independent rate-limit budget.

Repository layout

Path	Contents / reason
package.json	Workspace commands for starting and building both applications.
docker-compose.yml	Local pgvector-enabled PostgreSQL and Redis services.
.env.example	Required local configuration values; never commit the copied .env file.
backend/src/server.ts	Express tenant/key creation plus API-key-protected upload, list, search and Gemini chat routes.
backend/src/queue.ts and worker.ts	BullMQ queue and parse -> chunk -> Gemini embed -> pgvector ingestion worker.
backend/db/init.sql	Initial tables and HNSW vector index: users, tenants, API keys, documents, chunks.
frontend/src/main.tsx	Dashboard starter: tenant list, creation flow, one-time key display and copy action.
frontend/src/styles.css	Responsive dark dashboard styling for the starter UI.
README.md	Fast local setup and the boundary between current starter code and production work.

Local commands

Run these from the ChatStack directory. Docker Desktop must be running before the infrastructure command.

```
Copy-Item .env.example .env
Docker compose up -d
npm install
npm run dev
# API check: http://localhost:3000/health
# Dashboard: http://localhost:5173
npm run build
npm run worker -w backend
```

Configuration required before full functionality

Set GEMINI_API_KEY for embeddings and generated answers, then run the API and worker processes. Every document/search/chat request uses a private tenant API key; the server hashes that key at rest and derives tenant scope from it. Before public deployment, add rate limits, key rotation/revocation, streaming, user authentication, and S3 storage.

Phase 1 delivery sequence

Slice	Deliverable	Definition of done
1. Foundation	Compose, schema, app shells	Postgres with vector extension and Redis run locally; health endpoint answers.
2. Identity	Google OAuth + Redis sessions	Login, /me and logout work; dashboard routes require a session.
3. Tenant control plane	CRUD + secure API keys	Raw key shown once, hash stored, regenerate/revoke audited.
4. Documents	Multipart upload + local/S3-compatible storage	Tenant API key scopes every operation; status is visible.
5. Ingestion	Parse -> chunk -> embed -> store worker	A real PDF progresses pending to ready, retries safely on failure.
6. Retrieval and chat	Top-k search + streaming RAG	Citations/context are tenant scoped; first token is streamed.
7. Widget	Public config/chat route	Separate public key, origin policy and independent rate limits.
8. Product polish	Dashboard pages, errors, dark/light, responsive	Tenant journey is usable end to end.

Backend endpoint plan

Session authenticated: GET /auth/google, GET /auth/google/callback, GET /me, POST /auth/logout, and tenant create/list/detail/delete/key-rotation routes. Tenant API-key authenticated: document upload/list/detail/update/delete, POST /search and POST /chat. Public-key authenticated: widget config and widget chat. Cross-cutting routes include GET /health, centralized errors, request logging and Redis token-bucket limits.

Frontend plan

Build in the same order as the API slices: login/callback and UserContext; tenant dashboard; create-tenant dialog with one-time secret; tenant documents with drag/drop and status polling; chat test console; widget embed-snippet generator; settings and danger zone. Keep session-cookie requests separate from private bearer-key requests.

Phase 2 - AWS delivery plan

Deploy the static frontend to S3 behind CloudFront. Place the stateless Express API behind an Application Load Balancer on ECS Fargate. Run BullMQ as a distinct ECS worker service so document ingestion can scale independently. Use RDS PostgreSQL with pgvector and RDS Proxy, ElastiCache Redis, S3 for raw documents, and Secrets Manager for Gemini and database credentials.

Target operating envelope from the specification

Metric	Target
Active tenants	1,000
Indexed documents	100,000
Embedded chunks	Approximately 5,000,000
Chat load	20 RPS sustained; 100 RPS peak
Vector search p95	Under 150 ms
Chat p95 / first token	Under 2.5 s / under 600 ms target
Availability	99.5%

Operational decisions

Use HNSW on pgvector and scope queries by tenant_id. Cache hot API-key lookups and rate-limit counters in Redis. Add exponential retries and a dead-letter path for ingestion, make jobs idempotent per document, and add a circuit breaker around Gemini calls. Add a read replica when search load warrants it and partition chunks if large tenants dominate.

Recommended next implementation task

Start with database persistence and Google OAuth after supplying the OAuth credentials. That turns the visible dashboard into an authenticated tenant-control plane. Next, connect upload storage and the worker; do not expose search or chat until ingestion and tenant-key verification are proven against a real PDF.